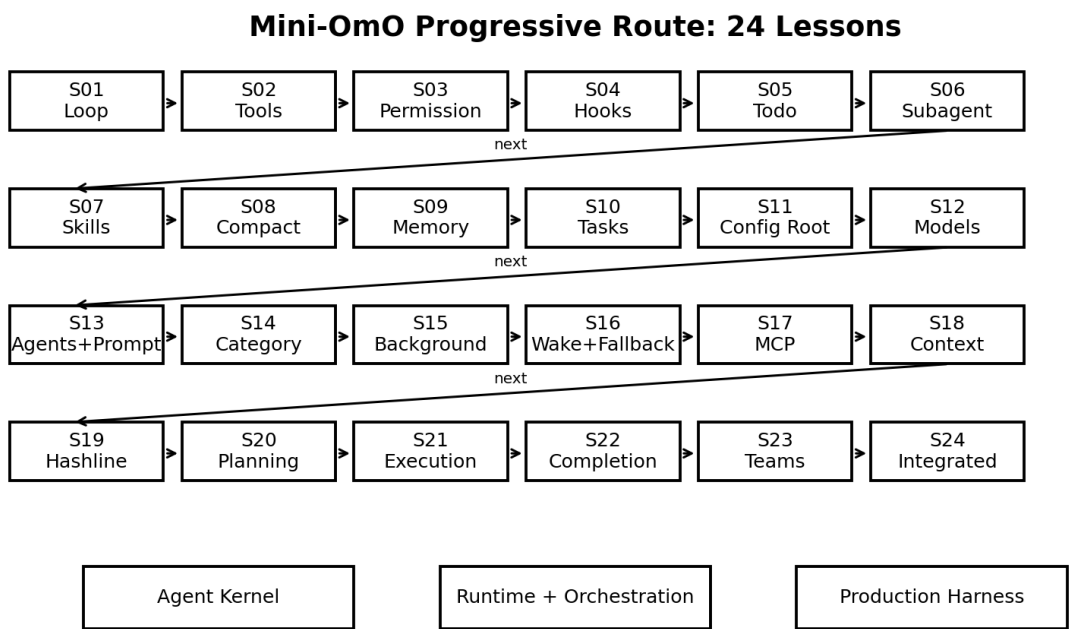


Mini-OmO

按照 learn-claude-code s01 -> s17 的方法 渐进式构建 oh-my-openagent Agent Runtime

完整 PDF 教程 | 2026-08-13 source snapshot



POSITIONING: 目标不是复制 OmO 现有几十个目录，而是学会从一个 30 行 Agent Loop 推导出 OmO 的关键架构：
Tool Registry、Policy Hooks、Dynamic Prompt、Task Delegation、BackgroundManager、Fallback、
Planning/Execution split、Team Mode 与 multi-harness adapters。

源码基线: learn-claude-code commit eb4307f4 (17-lesson refresh, 2026-08-12); oh-my-openagent dev commit 530c2c34 (2026-08-13). 教程中的 Mini-OmO 使用 Python 风格骨架帮助理解；真实 OmO 主要为 TypeScript/monorepo，本教程每课给出真实源码路径映射。

0. 如何使用这份教程

这份教程不是“读完就懂”的架构文章，而是一条建议实际提交 24 次 Git commit 的实现路线。每一课只引入一个主要概念；如果某课的测试没有通过，不要继续叠加下一层。

读者目标	建议路线
只理解 OmO 架构	先读 0-2 章，再读 S01/S04/S13/S15/S20-S24。
自己实现 Coding Agent	严格完成 S01-S18，再决定是否加入 Planning/Team。
做 Production Agent Runtime	全部完成，并重点执行附录中的失败注入、恢复、权限与并发测试。
阅读真实 OmO 源码	每课做完 Mini-OmO 后立即打开对应 OmO path，观察 production complexity 来自哪里。

CORE PRINCIPLE: learn-claude-code 最关键的教学思想不是“17 个功能”，而是让核心 loop 尽量稳定，让能力围绕 loop 增长。Mini-OmO 继续坚持同一原则。

Invariant: Keep the Agent Loop Small

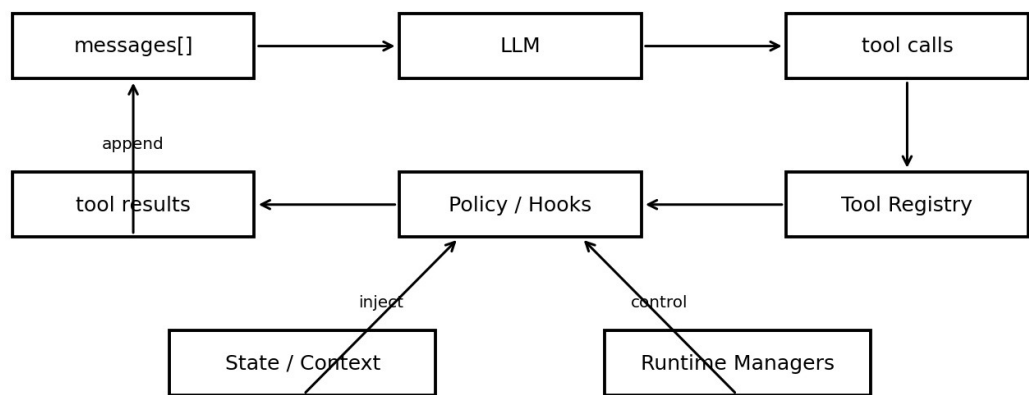


图 1. 能力增长时，核心 Agent Loop 尽量保持小而稳定。

1. 环境与仓库安装

建议把官方课程、真实 OmO 源码和自己的 Mini-OmO 放在同一个学习 workspace 下，但不要直接在真实 OmO 仓库里练习。

```
mkdir agent-runtime-lab && cd agent-runtime-lab

git clone https://github.com/shareAI-lab/learn-claude-code.git
git clone https://github.com/code-yeongyu/oh-my-openagent.git
git init mini-omo && cd mini-omo
python -m venv .venv
source .venv/bin/activate # Windows 使用 .venv\Scripts\activate
pip install anthropic python-dotenv pyyaml pytest
```

Mini-OmO 教学实现推荐沿用 learn-claude-code 的 Anthropic Messages/tool-use 形状，因为这样能把注意力放在 Agent Runtime，而不是 SDK 差异。真实 OmO 则运行在 OpenCode/Codex/Senpi host adapter 上。

```
.env
ANTHROPIC_API_KEY=...
MODEL_ID=claude-sonnet-4-6
# 也可以使用兼容 Anthropic API 的 provider:
# ANTHROPIC_BASE_URL=...
```

SECURITY: 不要把 API key、provider token、真实项目 secrets 提交进 Git。练习仓库中先写 .gitignore，再运行任何 Agent。

2. learn-claude-code -> Mini-OmO 的课程映射

learn-claude-code	原始概念	Mini-OmO 迁移
learn s01	Agent Loop	Mini-OmO S01: 保持核心循环不变量
s02	Tools	S02: Tool Registry
s03	Permission	S03: 三段式 Permission Pipeline
s04	Hooks	S04: Hook Bus / Control Plane
s05	TodoWrite	S05: Structured session state
s06	Subagents	S06: Subagent boundary
s07	Skill Loading	S07: Progressive Skill Loading
s08	Context Compact	S08 + S18: Context budget / rehydration
s09	Memory	S09: Durable memory
s10	Task System	S10 + S14: Task store / delegation
s11	Background Tasks	S15-S16: BackgroundManager + wake/fallback
s12	Cron	放入 Production Appendix; OmO 核心主线不依赖 cron
s13	Agent Teams	S23: Team Mode
s14	MCP	S17: MCP + SkillMCP
s15	Integrated Harness	S24: Composition Root / Plugin Interface
s16	Workflow Runtime	S20-S21: Planning -> execution workflow
s17	Goal Loop	S22: Completion Policy / continuation

OmO 特有而 learn-claude-code 没有单独作为课程中心讲解的部分，会插入 S11-S14、S18-S21：配置 composition root、模型解析、Agent registry、dynamic prompt、category delegation、hierarchical context、Prometheus/Atlas 分离。

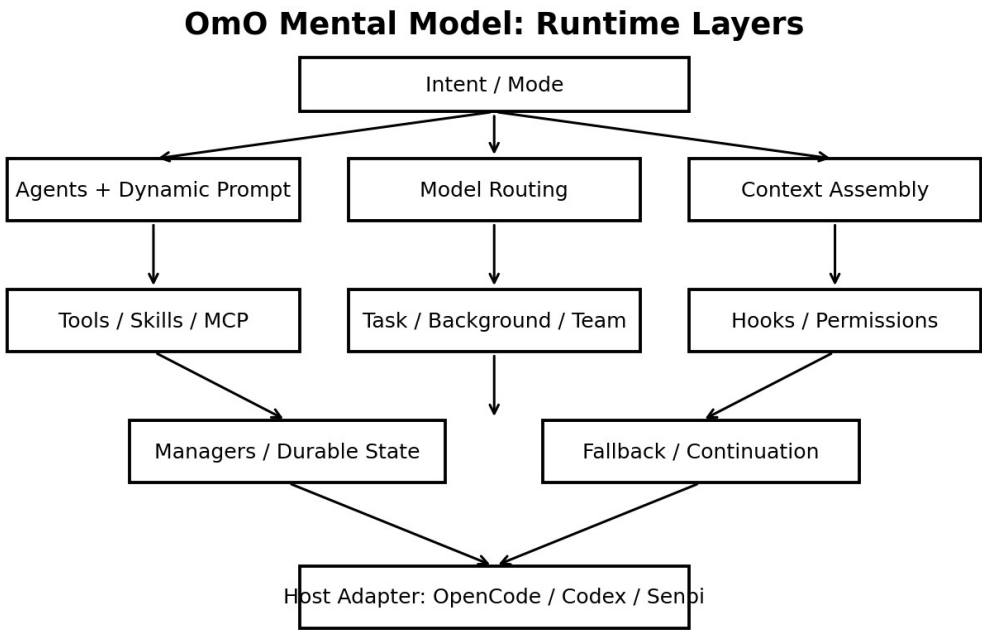


图 2. 真实 OmO 的复杂度主要集中在 Runtime control 和 orchestration，而不是基本 Tool Loop。

3. 最终 Mini-OmO 目录目标

```
mini_omo/  
  core/loop.py  
  tools/registry.py  
  policy/permissions.py  
  hooks/bus.py  
  state/todo.py  
  state/tasks.py  
  state/memory.py  
  skills/loader.py  
  context/compact.py  
  context/injector.py  
  models/catalog.py  
  agents/registry.py  
  agents/prompt_builder.py  
  routing/categories.py  
  delegation/task.py  
  runtime/background.py  
  runtime/parent_wake.py  
  runtime/fallback.py  
  mcp/manager.py  
  editing/hashline.py  
  orchestration/planning.py  
  orchestration/execution.py  
  orchestration/completion.py  
  team/runtime.py  
  adapters/opencode.py  
  app.py  
  tests/
```

前几课可以像 learn-claude-code 一样全部写在单文件中；到 S11 Composition Root 时再进行模块化重构。这样先理解行为，再处理包结构。

Part I. Agent Kernel - 保持 Loop 不变

S01. Agent Loop

INCREMENT: 只有一个 bash tool; while loop 把 tool_result 反馈给模型，直到模型不再 tool_use。

Know-Why: 为什么这一课现在出现

这是所有后续能力的“物理定律”。如果连 tool call/result pairing、消息追加顺序都没有吃透，后续任何 Hook、Subagent、Background 都只是更难调试的状态机。

核心设计

- messages 是唯一会话历史。
- assistant tool_use 必须先进入 history，再追加对应 tool_result。
- 停止条件首先是模型 stop_reason；后续 S22 才把“最终是否完成”从模型手里部分收回。

最小实现骨架

```
def agent_loop(messages):
    while True:
        resp = llm(messages=messages, tools=TOOLS)
        messages.append({"role": "assistant", "content": resp.content})
        if resp.stop_reason != "tool_use":
            return resp
        results = execute_tool_calls(resp.content)
        messages.append({"role": "user", "content": results})
```

动手实验

1. 让 Agent 运行 pwd / ls，并让结果影响下一轮回答。
2. 故意把 tool_result 不追加回 messages，观察 Agent 为什么“失忆”。
3. 记录每轮 messages 长度，理解 Tool Loop 如何膨胀 Context。

必须保持的不变量

- tool_use 与 tool_result 一一配对。
- Tool 执行失败也返回结构化结果，而不是抛出后终止整个会话。
- 核心 loop 不知道具体 bash 实现。

测试退出条件

- 一个需要两次 bash 才能回答的问题可正确完成。
- 一次 bash 报错后模型仍能修复并继续。

OmO 源码映射: learn-claude-code/s01_agent_loop/code.py; OmO 由 host 提供基础 loop，重点从 plugin-interface.ts 的生命周期接口观察如何包围 host loop。

S02. Tool Registry

INCREMENT: 从硬编码 bash 扩展到 read/write/edit/glob; 用 registry/dispatch map 代替 if-else。

Know-Why: 为什么这一课现在出现

Production Agent 的工具数量会快速增长。工具实现、schema、权限、feature gate 若混在 loop 里，任何新工具都会污染核心控制流。

核心设计

- ToolDefinition 与 handler 绑定。
- 路径工具先统一 safe_path。
- Tool Registry 负责 “暴露什么”，Tool implementation 负责 “怎么执行”。

最小实现骨架

```
class ToolRegistry:
    def __init__(self):
        self.defs = {}
        self.handlers = {}
    def register(self, definition, handler):
        self.defs[definition["name"]] = definition
        self.handlers[definition["name"]] = handler
    def execute(self, name, args):
        return self.handlers[name](**args)
```

动手实验

4. 加入 read_file / write_file / edit_file / glob。
5. 让 edit_file 要求 old_text 唯一匹配，避免模糊编辑。
6. 实现 disabled_tools，然后确认 schema 也从 LLM tool surface 消失。

必须保持的不变量

- 禁用工具 = 不暴露或明确拒绝，不能只在 Prompt 里说 “不要用”。
- 路径规范化先于读写。
- Tool 返回结果大小必须可控。

测试退出条件

- 未知工具返回可诊断错误。
- ../ 路径逃逸被阻止。
- 关闭 write 后模型工具列表里不再出现 write。

OmO 源码映射: packages/omo-opencode/src/create-tools.ts ; packages/omo-opencode/src/plugin/tool-registry.ts

S03. Permission Pipeline

INCREMENT: 在 Tool Execute 之前插入 hard deny -> contextual rule -> user approval 三道 gate。

Know-Why: 为什么这一课现在出现

Prompt 是概率约束，不是安全边界。Agent 说 “我不会删文件” 不等于 Runtime 不会执行危险命令。权限必须发生在执行路径上。

核心设计

- Gate 1: 不可覆盖 deny list。
- Gate 2: 根据 tool+args+workspace+agent role 匹配规则。
- Gate 3: 对可审批风险请求人类决定。
- 拒绝也要产生 tool_result，让模型知道为什么失败。

最小实现骨架

```
def authorize(call, ctx):
    if hard_deny(call): return Decision.deny("hard rule")
```

```

risk = match_rules(call, ctx)
if risk.requires_confirmation:
    return ask_user(call, risk.reason)
return Decision.allow()

```

动手实验

7. 阻止 `rm -rf /`、`sudo` 等不可审批命令。
8. `write` 到 `workspace` 外要求确认。
9. 给 `explore agent` 设置 `write=deny`，验证其 Prompt 无法绕过。

必须保持的不变量

- 权限决策不能由 MCP/server description 自行授予。
- 后台线程/无人值守任务不能弹出交互审批时，应 `fail closed`。
- 权限日志记录“谁、为什么、哪个参数”而不是只写 `deny`。

测试退出条件

- `deny list` 永远无法通过 `y` 覆盖。
- 后台任务遇到 `ask` 规则时默认拒绝。
- `read-only agent` 无法 `edit` 即使模型反复重试。

OmO 源码映射: `packages/omo-opencode/src/shared/agent-tool-restrictions.ts`; `packages/omo-opencode/src/plugin/tool-execute-before.ts`; `prometheus-md-only / team-tool-gating hooks`

S04. Hook Bus

INCREMENT: 把 permission、logging、output guard 从 loop 内硬编码迁移到固定生命周期 Hook。

Know-Why: 为什么这一课现在出现

Hook 是 OmO 的 Control Plane。它让核心 `loop/host` 保持稳定，同时在 `chat`、`tool`、`session`、`transform`、`continuation` 等时点注入横切策略。

核心设计

- 定义 `UserPromptSubmit` / `PreToolUse` / `PostToolUse` / `Stop` 四个教学事件。
- Hook 返回 `block/modify/continue` 的结构化决策。
- 注册顺序必须显式，因为早期 Hook 会改变后续 Hook 看到的数据。

最小实现骨架

```

class HookBus:
    def __init__(self): self._hooks = {}
    def on(self, event, fn): self._hooks.setdefault(event, []).append(fn)
    def emit(self, event, payload):
        for fn in self._hooks.get(event, []):
            decision = fn(payload)
            if decision and decision.block: return decision
        return None

```

动手实验

10. 把 S03 permission 改造成 `PreToolUse` hook。
11. 加入 `PostToolUse` output truncation hook。
12. 加入 `Stop` hook 统计工具调用，但先不强制继续。

必须保持的不变量

- Hook 不应偷偷吞掉异常；要记录 hook 名与阶段。
- 需要 dispose 的 Hook 必须有 lifecycle cleanup。
- Hook 顺序写测试，而不是靠维护者记忆。

测试退出条件

- Permission 行为与 S03 一致。
- 关闭某 Hook 后核心 loop 仍正常工作。
- 一个 Hook 抛错不会让所有 Hook 静默失效。

从教学版到 Production

- Production OmO 将 Hooks 分 Session / Tool Guard / Transform / Continuation / Skill tiers；不要在教学版第一天复制 50+ Hook。

OmO 源码映射: `packages/omo-opencode/src/create-hooks.ts`; `packages/omo-opencode/src/plugin/hooks/*`;
`packages/omo-opencode/src/hooks/AGENTS.md`

Part II. State, Subagents, Skills, Context

S05. Structured Todo / Session State

INCREMENT: 给模型一个 `todo_write` 工具，把“我接下来要做什么”从自然语言变成结构化 state。

Know-Why: 为什么这一课现在出现

长任务的问题通常不是模型完全不会做，而是进度状态只存在于自然语言 working memory。结构化 Todo 让 Runtime 可以检查、提醒、持久化或触发 continuation。

核心设计

- 状态限定 pending / in_progress / completed。
- 一次只允许一个 in_progress，减少任务注意力分叉。
- Runtime 记录 rounds_since_todo，长时间不更新就注入 reminder。

最小实现骨架

```
@dataclass
class Todo:
    content: str
    status: Literal["pending", "in_progress", "completed"]

class TodoManager:
    def update(self, items): validate(items); self.items = items
    def incomplete(self): return [x for x in self.items if x.status != "completed"]
```

动手实验

13. 让模型在多步骤重构前必须创建 Todo。
14. 连续三轮工具调用不更新 Todo 时注入 reminder。
15. 不要让 Todo 自动等于完成条件；S22 再引入独立 evaluator。

必须保持的不变量

- Todo 是 state，不是 Prompt 文本。
- 无效状态必须拒绝。
- Todo 更新失败不会丢失上一版本。

测试退出条件

- 超过 20 项被拒绝。
- 两个 in_progress 被拒绝。
- 任务完成后 incomplete() 为空。

OmO 源码映射: OmO todo-continuation-enforcer hook ; boulder / goal / task system related state modules

S06. Subagent Boundary

INCREMENT: 加入 `task(prompt)`：启动第二个 fresh messages loop，只把最终结果返回 parent。

Know-Why: 为什么这一课现在出现

Subagent 的价值不是“多一个人格”，而是隔离 Context、权限与工具面。Fresh context 可以避免主会话噪声，同时 worker 只需要知道一个窄任务。

核心设计

- Parent 与 Subagent 共享 workspace，但不共享 messages。
- Subagent 默认没有 task 工具，限制递归编排深度。
- 返回 parent 的应是高价值结果，而不是完整内部 transcript。

最小实现骨架

```
def run_subagent(prompt, profile):
    child = [{"role": "user", "content": prompt}]
    return run_loop(child,
        system=profile.system,
        tools=profile.tools_without_delegation,
        max_turns=30)
```

动手实验

16. 建一个 explore profile，只允许 read/glob/bash。
17. 父 Agent 让子 Agent 搜索 auth 实现，再由父 Agent 修改代码。
18. 测试 child context 不包含 parent 的无关历史。

必须保持的不变量

- Subagent 必须有 turn/spawn cap。
- 角色权限由 Runtime 提供，而不是仅靠角色 Prompt。
- 禁止无限 subagent -> subagent 递归。

测试退出条件

- 超过最大 turn 自动中止并返回诊断。
- read-only worker 无法 write。
- parent 能消费 worker 最终摘要。

OmO 源码映射: packages/omo-opencode/src/agents/* ; tools/delegate-task/subagent-resolver.ts ; Sisyphus-Junior tool restrictions

S07. Progressive Skill Loading

INCREMENT: 启动时只把 Skill name + description 放入 Prompt；真正需要时调用 load_skill(name) 读取 SKILL.md。

Know-Why: 为什么这一课现在出现

把所有领域知识塞进 system prompt 会消耗 Context 并制造互相干扰。Skill 是可发现、按需加载、可版本化的 capability package。

核心设计

- SKILL.md 使用 frontmatter name/description。
- 启动 scan 只构造 catalog。
- load_skill 返回全文，并记录本 session 已加载 skill。
- 同名 Skill 支持 scope priority。

最小实现骨架

```
class SkillLoader:
    def scan(self): ... # metadata only
    def catalog(self): return [(s.name, s.description) for s in self.skills]
    def load(self, name): return self.skills[name].content
```

动手实验

19. 创建 code-review 和 frontend 两个 Skill。
20. 验证不调用 load_skill 时完整正文不进入 context。
21. 实现 project > user > builtin 的同名覆盖。

必须保持的不变量

- Skill 发现与 Skill 注入分离。
- 未知 Skill 返回 available list。
- Skill 内若声明 MCP，不能直接把它当作已授权工具。

测试退出条件

- scope override 正确。
- 重复 load 同一 Skill 不重复注入。
- frontmatter 解析失败有清晰诊断。

OmO 源码映射: packages/omo-opencode/src/features/opencode-skill-loader/*; packages/skills-loader-core; features/skill-mcp-manager

S08. Context Budget + Compaction

INCREMENT: 在每次 LLM 调用前加入 output budget、micro-compact、archive、summary；遇到 prompt too long 再 reactive retry。

Know-Why: 为什么这一课现在出现

Context Window 是工作内存而不是日志仓库。Agent 工具输出特别容易把窗口占满；只做“最后一刻 summary”会丢失可恢复性与证据。

核心设计

- 大 Tool result 先落盘，只在 context 保留 path + preview。
- 旧 tool_result micro compact。
- 中段对话可 archive 到 transcript。
- 超过阈值时生成 summary，但保留 active request。

最小实现骨架

```
def prepare_context(messages):
    persist_large_tool_results(messages)
    snip_old_middle(messages)
    micro_compact_old_results(messages)
    if estimate(messages) > LIMIT:
        messages = summarize_and_rehydrate(messages)
    return messages
```

动手实验

22. 让 bash 输出 >30k 字符，验证正文落盘而不是全部进 prompt。
23. 制造 60+ messages，验证 transcript 可追溯。

24. 让 summary prompt 把 conversation 当数据而不是指令。

必须保持的不变量

- Compaction 前先保存可审计 transcript。
- summary 不是新的用户指令。
- tool_use/tool_result 配对不能被截断破坏。

测试退出条件

- 长输出后下一轮仍可调用模型。
- compaction 后 active user goal 存在。
- reactive prompt-too-long 只重试有限次数。

OmO 源码映射: anthropicContextWindowLimitRecovery ; preemptive-compaction ; tool-output-truncator ; compaction-context-injector

S09. Durable Memory

INCREMENT: 把跨会话稳定事实写入 .memory，并在新 session 检索相关记忆。

Know-Why: 为什么这一课现在出现

Conversation history 解决当前会话，Memory 解决跨会话状态。两者混在一起会导致 Context 永久增长，也会把一次性任务细节错误固化。

核心设计

- Memory 区分 user / feedback / project / reference。
- 只保存 persistent candidate；拒绝 “this session / for now” 等临时内容。
- Memory index 与 record 分开。
- Recall 必须有字符预算与相关性选择。

最小实现骨架

```
class MemoryStore:
    def write(self, fact):
        assert fact.scope == "persistent"
        assert not looks_temporary(fact)
        atomic_write(self.path(fact.slug), fact.to_markdown())
    def recall(self, query, budget): ...
```

动手实验

- 25. 记录“项目测试命令”作为 project memory。
- 26. 尝试记录“今天先不用测试”，验证被临时过滤。
- 27. 新进程中重载 memory 并注入相关事实。

必须保持的不变量

- Memory 是用户可编辑 durable state；恢复逻辑不得破坏用户修改。
- 存储前去重。
- 检索预算独立于完整 memory corpus 大小。

测试退出条件

- 进程重启后 memory 仍在。
- 重复记录不会生成多个近重复文件。

- 损坏单个 memory 不阻断整个 Agent 启动。

从教学版到 Production

- 先实现显式 durable memory，再研究 “active learning / dream consolidation”；后者属于高级自动化，错误成本更高。

OmO 源码映射: current OmO shared memory runtime / memory-core (v5 beta active-learning work) ; latest release notes: facts queue, people cards, dream consolidation

S10. Durable Task System

INCREMENT: 把 Todo 从 session 内状态升级为 .tasks 持久任务：id/status/owner/blockedBy。

Know-Why: 为什么这一课现在出现

Todo 适合 “我现在做到哪”，Task Store 适合 “系统中有哪些可恢复、可分配、带依赖的工作”。Background、Team、Workflow 都需要 durable task identity。

核心设计

- task_id 使用不可猜测/低冲突 id。
- create/claim/complete 明确状态机。
- blockedBy 形成 DAG 依赖。
- owner 为未来 multi-agent claim 做准备。

最小实现骨架

```
@dataclass
class Task:
    id: str; subject: str; status: str
    owner: str | None; blockedBy: list[str]

def claim(task_id, owner):
    with task_lock():
        t = load(task_id)
        assert t.status == "pending"
        assert all_done(t.blockedBy)
        t.status, t.owner = "in_progress", owner
        atomic_save(t)
```

动手实验

28. 建立 schema -> API -> tests 三个依赖任务。
29. claim tests 前应显示 blocked。
30. 完成 schema 后验证 API 自动变为 ready。

必须保持的不变量

- 任务 mutation 必须串行化。
- 写入使用 temp + rename，避免半文件。
- owner 规则要在 complete 时复核。

测试退出条件

- 依赖不存在视为 blocked。
- 重复 claim 只有一个成功。
- 进程重启后任务图可恢复。

OmO 源码映射: OmO task system / boulder-state ; team tasklist primitives ; workflow/journal patterns

Part III. OmO-Specific Cognition and Routing

S11. Config Layers + Composition Root

INCREMENT: 第一次模块化重构：配置加载、migration、Managers/Tools/Hooks 创建全部从 app.py 的 composition root 组装。

Know-Why: 为什么这一课现在出现

到这一阶段 side effects 已经很多。如果没有 Composition Root 和 dependency injection，测试 Background/MCP/Team 会被真实网络、进程、文件系统绑死。真实 OmO 的 createPluginModule() 就承担这个角色。

核心设计

- Config: user -> project ancestors -> harness -> profile。
- Composition Root 只负责 wiring，不写业务逻辑。
- 关键依赖可 overrides，形成 test seam。
- Manager 先于 Tool 创建，因为 Tool 依赖长生命周期 state。

最小实现骨架

```
def create_runtime(cfg, deps=DEFAULT_DEPS):
    managers = create_managers(cfg, deps)
    tools = create_tools(cfg, managers)
    hooks = create_hooks(cfg, managers)
    return Runtime(cfg, managers, tools, hooks)
```

动手实验

31. 把前 10 课单文件拆成 mini_omo package。
32. 测试用 FakeLLM/FakeClock/FakeProcessRunner 替换真实依赖。
33. 模拟 config migration conflict，确认 no-clobber。

必须保持的不变量

- 初始化顺序是依赖图，不能盲目 Promise.all/并行化。
- host 初始化 callback 中避免 re-enter host API 引起 deadlock。
- Config 默认值只在 merge 完成后应用。

测试退出条件

- 纯单元测试不需要 API key。
- migration dry-run 不写磁盘。
- manager dispose 可被测试调用。

OmO 源码映射: packages/omo-opencode/src/testing/create-plugin-module.ts ; create-managers.ts ; create-tools.ts ; create-hooks.ts ; plugin-config / config migration

S12. Model Catalog + Resolution Pipeline

INCREMENT: Agent 不再直接写死 MODEL_ID；引入 model catalog、provider availability、reasoning 与 ordered fallback chain。

Know-Why: 为什么这一课现在出现

Production Agent 的模型失败、provider 未连接、配额差异是常态。Routing 应成为 Runtime policy，而不是散落在每个 Prompt 或 Agent factory 中。

核心设计

- model catalog 把短名映射到 canonical model + reasoning。
- primary agent 可以尊重 UI-selected model。
- subagent 用角色自己的 fallback chain。
- 区分 proactive model resolution 与 reactive runtime fallback。

最小实现骨架

```
def resolve_model(req, env):
    candidates = [req.override, *req.role_chain, req.system_default]
    for c in candidates:
        if c and env.available(c):
            return Resolved(model=c.model, reasoning=c.reasoning,
                             provenance=c.source)
    raise NoModelAvailable()
```

动手实验

34. 准备两个 fake provider: primary unavailable, fallback available。
35. 记录 resolution provenance，最终输出为什么选到某模型。
36. 给 oracle 与 explore 配不同 fallback chain。

必须保持的不变量

- 模型可用性来自 host/cache，不信任 Prompt。
- resolution 是纯函数时更容易测试。
- fallback chain 中 per-model tuning 不应丢失。

测试退出条件

- primary unavailable 时选下一 rung。
- 全部不可用时明确失败，不偷偷换未知模型。
- reasoning override 优先级可预测。

OmO 源码映射: packages/model-core ; packages/omo-opencode/src/shared/model-resolution-pipeline.ts ; shared/model-requirements.ts

S13. Agent Registry + Dynamic Prompt

INCREMENT: 加入 sisypheus/oracle/librarian/explore/junior 等角色；Prompt 不再是静态巨串，而从当前 runtime capabilities 生成。

Know-Why: 为什么这一课现在出现

这是 OmO 最重要的 Context Engineering 原则之一：Prompt 不应该声明不存在的工具/Agent/Skill。Runtime 是真相，Prompt 是 Runtime 能力的投影。

核心设计

- AgentConfig 包含 mode/model/tools/permission/description。
- Agent registry 根据 disabled_agents 与 model availability 创建有效 Agent。
- Prompt builder 接收 availableAgents/Skills/Categories/Tools。

- 专长 Agent 使用明确 tool restriction。

最小实现骨架

```
def build_system(agent, runtime):
    return "

.join([
    identity_section(agent),
    delegation_table(runtime.available_agents),
    tool_guidance(runtime.tools_for(agent)),
    skill_catalog(runtime.available_skills),
    policy_section(agent),
])
```

动手实验

37. 禁用 oracle 后重新构建 Sisyphus prompt，确认 Delegation Table 不再提 oracle。
38. 禁用一个 Skill，确认 prompt catalog 同步变化。
39. 为 multimodal-looker 做 read-only allowlist。

必须保持的不变量

- Runtime capability -> Prompt；绝不反过来。
- 角色限制在 Runtime 重复执行。
- Dynamic prompt 每段最好是可单测的 builder。

测试退出条件

- 禁用 Agent 不可从 task resolver 访问。
- Prompt 不包含 disabled tool。
- agent sort/order 可稳定测试。

OmO 源码映射: packages/omo-opencode/src/agents/builtin-agents.ts ; agents/dynamic-agent-* ; packages/prompts-core ; shared/agent-tool-restrictions.ts

Dynamic Prompt = Projection of Runtime Capabilities

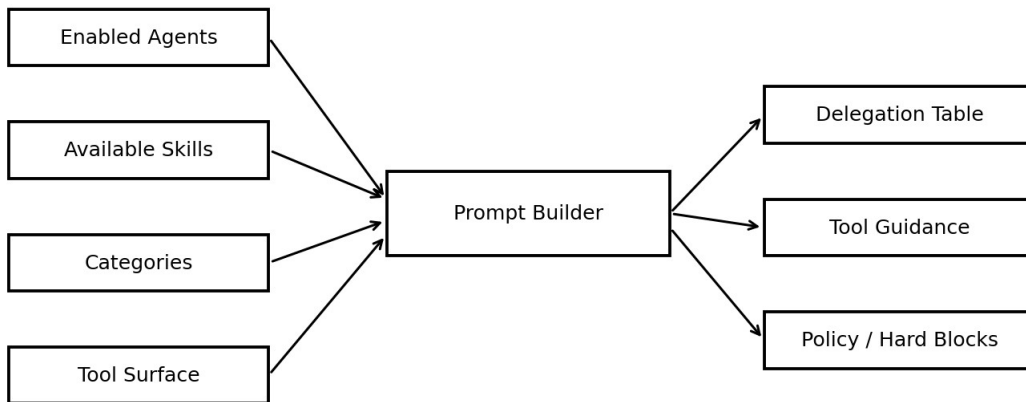


图 3. Dynamic Prompt 不创造能力，只把已存在的 Runtime capability 组织成模型可理解的上下文。

S14. Category Router + Task Delegation Engine

INCREMENT: 把 `task(subagent_type=...)` 扩展为 `task(category=...)`；Category 描述工作意图并解析到 `model + Sisyphus-Junior`。

Know-Why: 为什么这一课现在出现

如果所有任务都直接绑定模型或 Agent 名，业务意图会与具体 provider 耦合。Category 将“这是什么工作”与“谁/什么模型执行”解耦。

核心设计

- `category` 与 `subagent_type` 互斥。
- `Category` -> `config/model` -> Junior executor。
- `Subagent type` -> `named specialist`。
- `Skill resolver` 把 `load_skills` 注入任务 `prompt`。
- 执行器路由 `sync` 或 `background`。

最小实现骨架

```
def resolve_task(args, runtime):
    if bool(args.category) == bool(args.subagent_type):
        raise InputError("choose exactly one")
    if args.category:
        model = runtime.categories.resolve(args.category)
        agent = runtime.agents["sisyphus-junior"]
    else:
        agent = runtime.agents[args.subagent_type]
        model = runtime.models.for_agent(agent)
    return TaskPlan(agent, model, runtime.skills.resolve(args.load_skills))
```

动手实验

40. 实现 quick/deep/writing 三个 category。
41. task(category=deep, load_skills=[...]) 生成 worker prompt。
42. 尝试同时传 category 和 subagent_type，必须拒绝。

必须保持的不变量

- Category 是 routing policy，不是新 Persona。
- Junior 默认不能再次 delegate。
- 模型不可用时走统一 model resolution，而不是 category 自己写 fallback 逻辑。

测试退出条件

- 未知 category 给出可用列表。
- Category dispatch 一定走 Junior。
- read-only specialist 仍遵守自身权限。

OmO 源码映射: packages/omo-opencode/src/tools/delegate-task/* ; packages/delegate-core ; shared/category/model requirements

Part IV. Runtime Managers - 从同步工具到异步 Agent OS

S15. BackgroundManager + Concurrency

INCREMENT: 把同步 subagent/task 扩展为 durable background lifecycle: pending -> queued -> running -> polling -> terminal。

Know-Why: 为什么这一课现在出现

真正的 Multi-Agent 价值来自并行。简单 `asyncio.create_task` 不能解决 provider 限流、取消、孤儿任务、结果恢复与 parent 通知。

核心设计

- BackgroundTask 有稳定 id/status/attempt/session id。
- Concurrency key 按 provider/model，而不是只有全局 semaphore。
- 队列 FIFO 获取 slot。
- terminal 状态释放 slot。
- poller 与 session event 共同判断生命周期。

最小实现骨架

```
class BackgroundManager:
    async def launch(self, spec):
        task = self.store.create(spec, status="pending")
        key = concurrency_key(spec.model)
        await self.concurrency.acquire(key)
        try:
            return await self.spawner.start(task)
        except Exception:
            self.concurrency.release(key)
            raise
```

动手实验

43. 同一 provider/model limit=2，一次 launch 5 个 worker。
44. 验证第三个开始排队，前两个结束后 FIFO 启动。
45. 取消一个 running task，slot 必须释放。

必须保持的不变量

- 每个 terminal path 都释放并发 slot。
- 任务状态变化可查询。
- 进程退出要 best-effort 清理子进程/session。

测试退出条件

- 无 slot leak。
- cancel queued 与 cancel running 都正确。
- 同 provider 不同 model 可分别限流。

OmO 源码映射: `packages/omo-opencode/src/features/background-agent/*`; BackgroundManager; ConcurrencyManager; task-poller.ts

Background Runtime: State + Concurrency + Completion

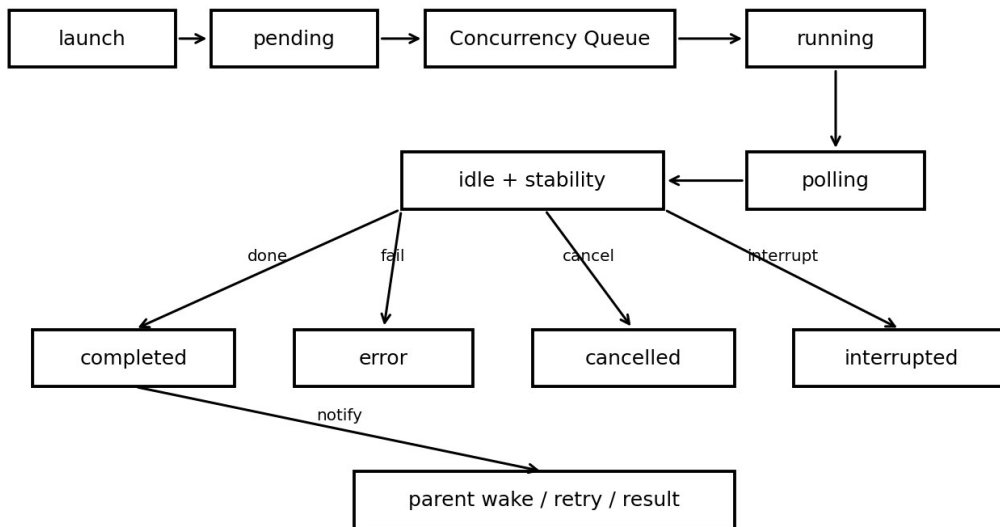


图 4. BackgroundManager 的核心不是 thread，而是任务生命周期与资源管理。

S16. Completion Detection + Parent Wake + Runtime Fallback

INCREMENT: 后台任务不只“跑完”：加入 idle+stability completion、parent wake、error classification 与 reactive fallback retry。

Know-Why: 为什么这一课现在出现

LLM session 短暂 idle 不一定完成；provider error 也不应该直接等于任务失败。Production runtime 要区分完成、暂停、可重试错误与不可恢复错误。

核心设计

- 完成 = session idle signal + 一段时间 message stability。
- Task result 注入 parent 的高价值 system/context message。
- 错误分类映射 rate-limit/provider/context/permission 等。
- Runtime fallback 在真实 API error 后切下一模型 rung。

最小实现骨架

```
async def maybe_complete(task):
    if task.idle_seen and stable_for(task, seconds=10):
        result = await fetch_final_result(task)
        await parent_wake.notify(task.parent_id, result)
        task.status = "completed"

async def on_provider_error(task, err):
    kind = classify(err)
    if retryable(kind):
        return await retry_with_next_model(task)
    fail(task, kind)
```

动手实验

46. 制造一次 transient provider failure，第二 rung 成功。
47. 制造短暂 idle 后又有新 message，确认不会提前 completed。
48. parent 主循环继续工作时后台完成，再在下一轮读取结果。

必须保持的不变量

- Parent wake 不把整个 child transcript 灌回主 Context。
- retry 有 attempt cap。
- Fallback 不绕过权限错误。

测试退出条件

- 短 idle 不误判。
- 可重试错误切 model，permission denied 不切模型。
- parent notification exactly once。

OmO 源码映射: features/background-agent/parent-wake-notifier.ts ; fallback-retry-handler.ts ; session-idle-event-handler.ts ; hooks/runtime-fallback ; hooks/model-fallback

S17. MCP + SkillMCP Manager

INCREMENT: 让 Runtime 可以连接外部 MCP server，动态 tools/list -> tool registry，并支持 Skill-scoped MCP 生命周期。

Know-Why: 为什么这一课现在出现

MCP 让工具生态可扩展，但“server 描述自己是 read-only”不能成为授权依据。MCP discovery、tool naming、host policy、lifecycle isolation 必须分开。

核心设计

- MCPManager 负责 server connection 与 tools/list。
- 工具名 namespace 化，例如 mcp__docs__search。
- Host policy 决定 allow/confirm/deny。
- Skill MCP 以 session:skill:server 做隔离。

最小实现骨架

```
class MCPManager:
    async def connect(self, name, server):
        tools = await server.list_tools()
        for t in tools:
            public = f"mcp__{norm(name)}__{norm(t.name)}"
            registry.register(adapt_schema(public,t),
                             lambda **a: self.call(name,t.name,a))
```

动手实验

49. 连接一个 mock docs MCP，动态新增 search/get_version。
50. 再连接 deploy MCP，把 trigger 标成 confirm。
51. 两个 session 加载同名 Skill MCP，状态不可串线。

必须保持的不变量

- Authorization 来自 host config。

- MCP disconnect/dispose 不留孤儿进程。
- 外部 schema 先 normalize 后才暴露给模型。

测试退出条件

- 名称冲突能诊断。
- server 崩溃不让主 Agent 整体挂死。
- 危险 MCP tool 正确触发 permission。

OmO 源码映射: plugin-handlers/mcp-config-handler.ts ; features/skill-mcp-manager ; built-in MCP tiers / .mcp.json loader

S18. Hierarchical Context Injection + Rehydration

INCREMENT: 在 **compaction** 之外，再加入 **AGENTS.md/README/rules** 的按目录注入，以及 **compaction** 后 **state rehydration**。

Know-Why: 为什么这一课现在出现

Context Engineering 不是“更多上下文”，而是“当前一步正确的上下文”。同一 repo 不同目录往往有不同规则；compaction 后还必须恢复目标、Todo、计划与局部规则。

核心设计

- 文件操作前向上查找 relevant AGENTS.md/README。
- rules 根据 glob/context 条件注入。
- Context injector 做去重与优先级。
- compaction 后 rehydrate goal/todo/plan/rules，而不是只相信 summary。

最小实现骨架

```
def context_for(path, session):
    layers = discover_agent_files(path)
    rules = rule_engine.match(path, session.mode)
    return dedupe([*layers, *rules, session.goal, session.todo_state])

def after_compaction(session):
    return rehydrate(session.durable_state)
```

动手实验

52. 根目录和 src/components 各放一个 AGENTS.md，读取组件文件时验证层级上下文。
53. 让 compaction 发生后继续任务，确认 Todo/goal 仍在。
54. 规则只对 *.ts 命中。

必须保持的不变量

- 局部规则不应无条件污染全局 Prompt。
- re-hydration 从 durable state 读取，而非只从模型 summary。
- 避免同一 AGENTS.md 重复注入。

测试退出条件

- 目录切换时 context 正确变化。
- compaction 后关键 state 不丢。
- 规则匹配单元测试覆盖 include/exclude。

OmO 源码映射: directory-agents-injector ; directory-readme-injector ; rules-injector ; compaction-context-injector ; compaction-todo-perserver

S19. Hashline Edit + File Safety

INCREMENT: 把普通 `old_text edit` 升级为 `LINE#HASH anchored edit`；执行前验证引用仍对应当前文件。

Know-Why: 为什么这一课现在出现

Agent 经常“读过后再编辑”，期间文件可能被其他 Agent/用户改变。Hashline 相当于把数据库 optimistic concurrency control 引入代码编辑。

核心设计

- Read 输出每行 `line_number#short_hash`。
- Edit 使用 `anchor + op replace/append/prepend`。
- 执行前重新计算 hash；不匹配拒绝 stale edit。
- 多编辑 bottom-up 应用，避免行号漂移。

最小实现骨架

```
def apply_edit(file, edit):
    lines = read_lines(file)
    line_no, expected = parse_anchor(edit.pos)
    if hash_line(lines[line_no-1]) != expected:
        raise StaleReference()
    apply_bottom_up(lines, normalize(edit))
    atomic_write(file, lines)
```

动手实验

55. 读文件得到 anchors，外部修改一行后再用旧 anchor edit，必须拒绝。
56. 同一文件同时 replace line10 与 line100，验证 bottom-up。
57. 写现有文件前要求 read evidence。

必须保持的不变量

- stale reference fail closed。
- 换行符/BOM 等 canonicalization 要稳定。
- 文件写入尽量 atomic。

测试退出条件

- 旧 hash 被拒绝。
- 新 hash edit 成功。
- overlapping ranges 被拒绝。

OmO 源码映射: `packages/omo-opencode/src/tools/hashline-edit/*`; `hashline-read-enhancer`; `write-existing-file-guard`; `bash-file-read-guard`

Part V. Orchestration - 从 Worker 到可验证 workflow

S20. Planning Pipeline: Prometheus + Metis + Momus + Oracle

INCREMENT: 引入独立 **planning mode**: 先澄清需求、研究、生成计划，再做 gap/review; Planner 不直接改代码。

Know-Why: 为什么这一课现在出现

复杂任务把“想清楚”和“执行”放在一个长 Context 中容易产生 cognitive drift。Planning/Execution 分离让计划成为 durable artifact，并允许独立 reviewer 校验。

核心设计

- Prometheus 负责 interview + plan artifact。
- Metis 在写计划前做 gap analysis。
- Momus/Oracle 独立 review，可形成 dual-review loop。
- Planner 写权限限制在计划 markdown 范围。

最小实现骨架

```
def planning_workflow(request):
    context = research(request)
    clarified = interview_until_clear(request, context)
    gaps = metis.review(clarified)
    plan = prometheus.write_plan(clarified, gaps)
    while high_accuracy and not reviewers_approve(plan):
        plan = prometheus.fix(plan, reviewer_feedback)
    return plan
```

动手实验

58. 给一个模糊“重构认证系统”需求，Planner 必须先提边界/测试问题。
59. Metis 找出缺失 acceptance criteria。
60. 尝试让 Planner 编辑 .py 文件，Runtime 拒绝。

必须保持的不变量

- Plan 是 durable state，不只是聊天文本。
- Reviewer 与 author 独立。
- review rejection 必须携带可修复 evidence。

测试退出条件

- Planner 不写源码。
- 高精度模式只有 reviewers 均通过才 handoff。
- 计划引用的关键文件存在。

OmO 源码映射: Prometheus agent/config builder ; Metis/Momus/Oracle agents ; prometheus-md-only hook ; docs/guide/orchestration.md

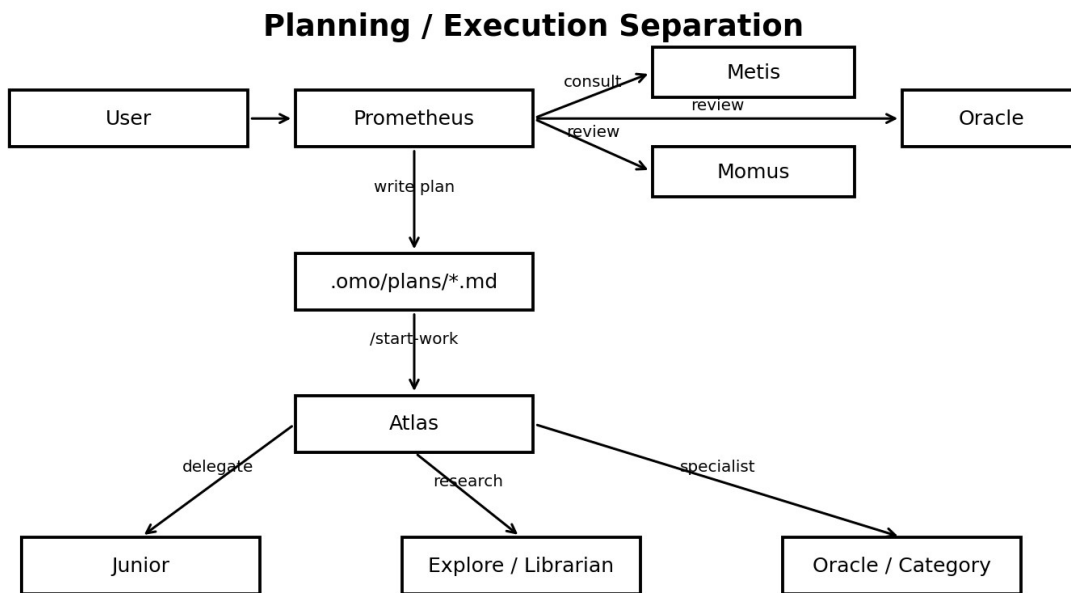


图 5. Planning 和 Execution 分离，是 OmO 复杂任务编排的核心结构。

S21. Execution Pipeline: Atlas + Sisypheus-Junior

INCREMENT: 用 `/start-work` 把 plan 交给 Atlas；Atlas 只做编排/验证，把写代码的 task 分给 Junior 或 specialist。

Know-Why: 为什么这一课现在出现

Orchestrator 如果自己做所有实现，会在“计划、搜索、编码、验证”之间反复切换上下文。Atlas 把复杂度压缩为 task selection、wisdom accumulation、verification。

核心设计

- Atlas 读取 plan，构造可执行 task。
- Category task 交给 Junior；specialist task 直达 oracle/explore/librarian。
- 每个 worker 返回 result + learnings。
- Atlas 把 learnings 传给后续 worker，并独立 verification。

最小实现骨架

```

for task in plan.tasks:
    spec = choose_worker(task)
    result = delegate(spec, background=False)
    wisdom.update(extract_learnings(result))
    verify(task, result)
    durable_plan.mark_done(task.id)
  
```

动手实验

61. 把 3 个互相依赖的计划项交给 Junior。
62. 第二个 worker 收到第一个的 conventions/gotchas。
63. worker 声称“tests pass”但没有 evidence 时，Atlas 自己跑测试验证。

必须保持的不变量

- Orchestrator 不应信任 worker 自报完成。
- Junior 不继续 delegate，限制编排深度。
- 计划 checkbox/state 只能由受控流程更新。

测试退出条件

- 每个 task 都有 verification evidence。
- 失败任务可重试而不重跑已完成项。
- wisdom 不包含 secrets/超大 transcript。

OmO 源码映射: Atlas agent / atlas hook ; Sisyphus-Junior agent ; task/delegate engine ; notepad / boulder plan state

S22. Completion Policy: Continuation + Goal Loop

INCREMENT: 模型停止不再等于任务完成；Stop hook 调独立 evaluator 判断 completion condition，未满足则继续同一 loop。

Know-Why: 为什么这一课现在出现

LLM 的 stop_reason 只表示“这一轮不再调用工具”。Production Agent 需要 Runtime 级 completion policy: Todo 未完成、计划未完成、goal evidence 缺失时不能简单退出。

核心设计

- GoalState 保存 condition/iterations/token baseline。
- 独立 evaluator 无工具，只看 transcript evidence。
- StopDecision: allow / block-and-continue / impossible。
- 设置 block cap，防止无限 boulder loop。

最小实现骨架

```
def on_stop(session):
    if session.goal:
        ev = evaluator.check(session.goal, transcript(session))
        if ev.ok: return ALLOW
        if ev.impossible: return FAIL(ev.reason)
        if session.goal.iterations >= MAX_BLOCKS: return FAIL("cap")
        return CONTINUE(ev.reason)
    if session.todos.incomplete():
        return CONTINUE("unfinished todos")
    return ALLOW
```

动手实验

64. goal="pytest tests exits with code 0"，没有测试结果时 evaluator 应 block。
65. 有真实 exit code=0 evidence 后允许完成。
66. 设置无法实现条件，evaluator 返回 impossible 后停止。

必须保持的不变量

- Evaluator 无工具且把 transcript 当数据。
- 不能凭 worker 说“完成了”就 ok，要求 evidence。
- Continuation 有次数/token cap。

测试退出条件

- 没有 goal 时普通任务仍能结束。
- goal 不满足会重新进入 loop。
- block cap 到达后输出明确未完成原因。

OmO 源码映射: goal hook (replaces legacy ralph loop) ; todo-continuation-enforcer ; atlas continuation ; compaction autocontinue

S23. Team Mode

INCREMENT: 从 parent-child delegation 升级到 lead + persistent members + shared mailbox + shared task list + optional worktrees。

Know-Why: 为什么这一课现在出现

Background worker 适合独立任务；Team Mode 只有在成员需要长期身份、互相通信、竞争 claim shared tasks 时才值得引入。它本质上是一个小型分布式系统。

核心设计

- TeamSpec 先验证 agent eligibility。
- sessionID -> team/member registry 解决 spawn race。
- Mailbox 使用 deferred ack。
- task claim 使用 atomic lock。
- state temp+rename。
- 禁止 nested team。

最小实现骨架

```
async def team_create(spec):
    validate_members(spec)
    state = init_state_atomic(spec)
    for m in spec.members:
        sid = await spawn_member(m)
        register_team_session(sid, m) # immediately
    return state

def claim_shared_task(task_id, member):
    with cross_process_lock(task_id):
        assert task.pending and dependencies_done(task)
        task.owner, task.status = member, "claimed"
        atomic_save(task)
```

动手实验

67. 两个 member 同时 claim 同一个 task，只有一个成功。
68. lead 发送 broadcast 与单播，member 分别 ack。
69. lead 退出后测试 orphan handling。
70. 启用 per-member worktree，避免同时编辑冲突。

必须保持不变量

- spawn race lookup 优先 registry。
- 所有共享 state mutation 加锁。
- read-only/plan-only agent 在 TeamSpec parse 阶段拒绝。
- nested team 禁止。

测试退出条件

- 并发 claim 无双写。
- message 不丢且 ack 可重放。
- team_delete 清理 state/worktree/panes。

OmO 源码映射: packages/omo-opencode/src/features/team-mode/* ; packages/team-core ; team-session-events hooks ; team_* tool family

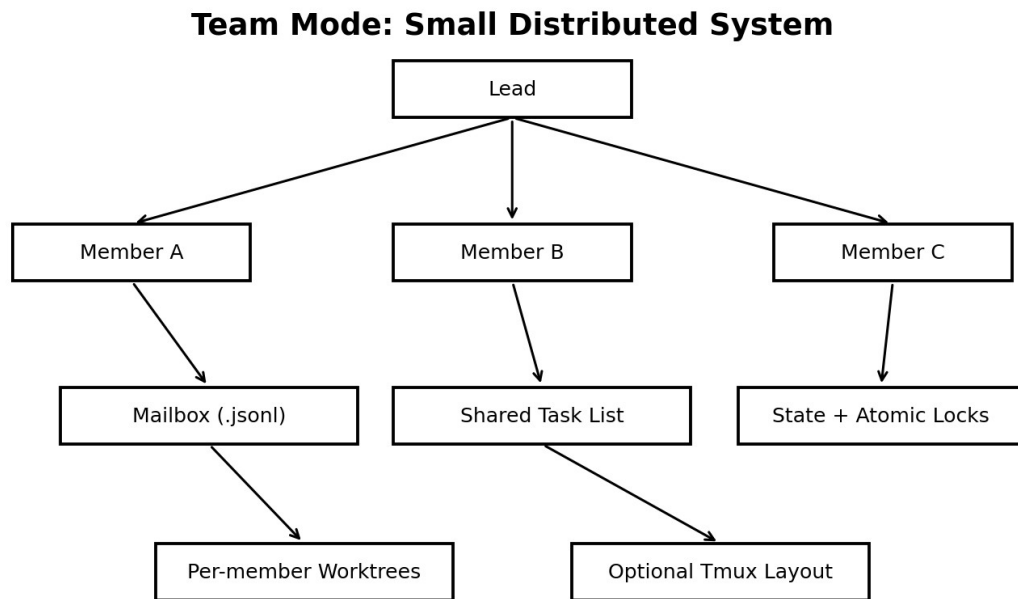


图 6. Team Mode 只有在需要共享协议与长期 member identity 时才比 background task 更合适。

Part VI. Integrated OmO Harness and Productionization

S24. Integrated Plugin Runtime + Multi-Harness Adapter

INCREMENT: 把前 23 课组装成 **Composition Root**，并把 **host-specific lifecycle** 收敛到 **adapter/plugin-interface**；共享逻辑逐步抽到 ***-core**。

Know-Why: 为什么这一课现在出现

真实 OmO 的终点不是“一个巨型 agent.py”，而是共享纯逻辑 + harness adapter + lifecycle managers。OpenCode、Codex、Senpi 的差异应被 adapter 吸收，而不是污染 task/model/team core。

核心设计

- Startup: migration/config -> runtime services -> managers -> tools -> hooks -> plugin interface。
- Managers 持有长生命周期资源；Tools 是 LLM-callable surface；Hooks 是 control plane。
- dispose 统一清理 background/MCP/tmux/hooks。
- 纯逻辑逐步抽为 model-core/delegate-core/team-core/skills-loader-core 等。

最小实现骨架

```
def create_plugin(ctx, cfg, deps):
    state = create_model_cache_state()
    managers = create_managers(ctx, cfg, state, deps)
    tools = create_tools(ctx, cfg, managers)
    hooks = create_hooks(ctx, cfg, managers, tools.skills)
    interface = create_plugin_interface(ctx, cfg, managers, tools, hooks)
    return Plugin(interface=interface,
                  dispose=lambda: dispose_all(managers, hooks))
```

动手实验

71. 用 FakeHost 跑完整 startup -> tool -> hook -> dispose。
72. 把 model resolver 抽成不依赖 OpenCode 的 pure core。
73. 写第二个 MockHarnessAdapter，证明 task/category/permission core 可重用。

必须保持的不变量

- Adapter boundary 清晰：共享 core 不 import host SDK。
- dispose 幂等。
- startup 不允许隐藏的 host re-entry deadlock。
- 配置与 capabilities 必须在 Prompt/Tool registry 之前解析完成。

测试退出条件

- unit tests 可不启动真实 host。
- 重复 dispose 不报错。
- OpenCode adapter 关闭某 feature 后 Tool/Prompt/Hook 三处一致。

从教学版到 Production

- 在真实 OmO 中，最新 dev 正持续把 OpenCode 特定实现拆成 core + adapter。读源码时始终问：这是 domain implementation，还是 harness shim？

OmO 源码映射: packages/omo-opencode/src/testing/create-plugin-module.ts ; plugin-interface.ts ; create-managers.ts ; create-tools.ts ; create-hooks.ts ; packages/*-core

Composition Root and Multi-Harness Extraction

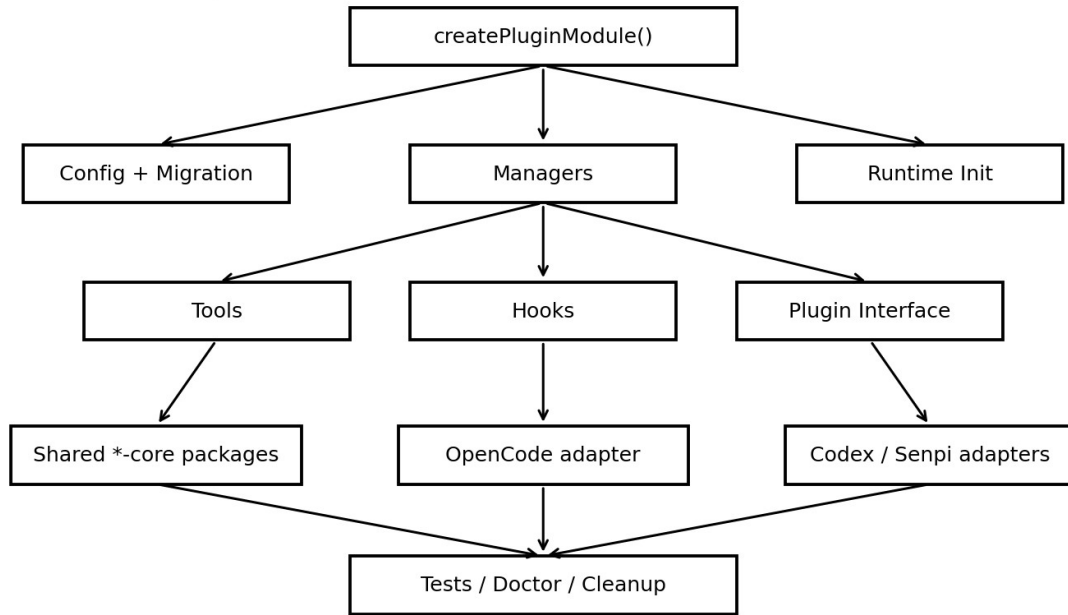


图 7. Production 终局：Composition Root 组装长生命周期 Manager、Tool surface、Hook control plane，再通过 adapter 对接 host。

4. 建议的 24 次 Git Commit 实现计划

Commit	建议 message
01	feat: minimal tool-use loop
02	feat: tool registry and safe paths
03	feat: permission decision pipeline
04	refactor: lifecycle hook bus
05	feat: structured todo state
06	feat: isolated subagent runner
07	feat: progressive skill loader
08	feat: context budget and compaction
09	feat: durable memory store
10	feat: persistent task DAG
11	refactor: composition root and DI seams
12	feat: model catalog and resolution pipeline
13	feat: agent registry and dynamic prompt builder
14	feat: category-based task delegation
15	feat: background task manager and concurrency
16	feat: parent wake and runtime fallback
17	feat: MCP discovery and skill-scoped MCP
18	feat: hierarchical context and rehydration
19	feat: hash-anchored editing guards
20	feat: planning workflow and reviewers
21	feat: execution orchestrator and junior workers
22	feat: goal completion evaluator and continuation
23	feat: team mailbox tasklist and worktrees
24	refactor: integrated host adapter and cleanup

WORKFLOW: 每一个 commit 都应该可以单独运行上一课的回归测试。不要把“能工作”与“架构更漂亮”放在同一个大 commit 里，否则你很难知道行为改变来自哪里。

5. Test Matrix - 课程是否真正完成

维度	最低测试	Production 加强
Loop protocol	tool_use/result pairing; tool error recovery	property tests for mixed text/tool blocks
Filesystem	path escape; unique edit match	symlink escape; atomic write; concurrent edits
Permission	hard deny; ask; agent deny	non-interactive fail-closed; audit log
Hooks	order; disable; error isolation	disposal; latency budget; mutation conflicts
Context	large output; compact; active goal retained	token estimator; crash recovery; transcript integrity
Memory	restart persistence; dedupe	consolidation conflict; user edit preservation
Tasks	dependency DAG; owner state	cross-process lock; journal recovery
Models	unavailable primary -> fallback	provider outage chaos; provenance
Delegation	category/subagent exclusivity	spawn caps; timeout; retry budget
Background	queue/cancel/slot release	orphan cleanup; parent wake exactly-once
MCP	schema adapt; namespace	server crash; permissions; process cleanup
Completion	goal evidence; continuation cap	adversarial transcript; impossible goals
Team	atomic claim; mailbox ack	orphan lead; restart/resume; worktree cleanup
Composition	fake host startup/dispose	duplicate plugin detection; migration interruption

6. 真实 oh-my-openagent 源码阅读顺序

完成 Mini-OmO 后，建议按 “composition -> control plane -> stateful runtime -> cognition -> advanced orchestration” 的顺序读真实 TypeScript，而不是先读最长的 Sisyphus Prompt。

- 1) packages/omo-opencode/src/index.ts + testing/create-plugin-module.ts - 找到 composition root。
- 2) create-managers.ts - 区分 long-lived managers 与一次性 tools。
- 3) create-tools.ts + plugin/tool-registry.ts - 看 feature gates 如何决定真实 Tool surface。
- 4) create-hooks.ts + plugin/hooks/* + hooks/AGENTS.md - 看 control plane 分 tier。
- 5) plugin-interface.ts - 理解 OpenCode adapter boundary。
- 6) agents/builtin-agents.ts + dynamic-agent-* - 看 Runtime 如何生成 Prompt。
- 7) tools/delegate-task/* + packages/delegate-core - 看 category/subagent/model/skill resolution。
- 8) features/background-agent/* - 读任务生命周期、并发、completion、parent wake。
- 9) shared/model-resolution-pipeline.ts + packages/model-core - 读 model policy。
- 10) features/opencode-skill-loader/* + skill-mcp-manager - 读 progressive disclosure。
- 11) tools/hashline-edit/* - 读 optimistic concurrency editing。
- 12) features/team-mode/* + packages/team-core - 按分布式系统读。
- 13) plugin-handlers/* - 读 6-phase config application。
- 14) 最后再读 prompts-core / Sisyphus / Prometheus / Atlas prompt variants。

7. 从 learn-claude-code 到 OmO：真正发生了什么变化

阶段	教学 Harness	OmO Production Runtime
Tool	dict handlers	动态 registry + feature gates + schema normalization + MCP
Permission	几个字符串规则	agent permissions + tool guards + role restrictions + host gate
Hook	4 个事件	多 tier lifecycle control plane
Subagent	fresh messages loop	Agent registry + category routing + model policy + background sessions
Context	compact summary	hierarchical injection + output budget + compaction rehydration
State	Todo / JSON task	Boulder/Goal/Task/Team/Memory durable states
Background	thread/queue	concurrency by provider-model + poller + fallback + parent wake
Completion	model stops	continuation hooks + goal/evidence policy
Team	threads + shared files	eligibility + registry + mailbox + locks + worktrees + lifecycle
Architecture	single file	composition root + managers/tools/hooks + shared core + adapters

KNOW-WHY: OmO 的主要工程对象已经不是 Prompt。Prompt 是 Runtime 的一个视图；Tool 是 Runtime 的一组 affordance；Subagent 是 Runtime 的一种执行策略。真正决定可靠性的是 State、Policy、Context、Routing、Lifecycle、Recovery、Verification。

8. 10 个最容易走错的实现方向

Anti-pattern	替代策略
把所有规则塞进 system prompt	无法 deterministic enforce；优先 Runtime permission/hook。
每新增功能就改核心 while loop	最终变成不可测试的大状态机；优先 lifecycle hooks/managers。
所有工具永久暴露	增加误选和 schema context；按 feature/agent/skill 过滤。
Subagent 可以无限再 delegate	递归深度和成本不可控；Junior/worker 禁止 task。
全局并发 semaphore	忽略 provider/model 限流差异；按 concurrency key。
idle 就标记 background completed	会 premature completion；结合 stability/evidence。
Compaction 只做 summary	Todo/Goal/Plan/rules 会丢；做 state rehydration。
Agent 自报 tests pass 就相信	Orchestrator 应运行独立 verification。
Team 作为默认并行方式	协议复杂度高；80% 场景先用 background workers。
一开始复制真实 OmO 50+ hooks	学习曲线爆炸；从 4 个 hook points 增长到你真实需要的控制面。

9. Capstone Labs

Lab A - 代码库认证调查

Sisyphus parent 并行 launch Explore + Librarian，主线程继续读取项目；结果回来后合并证据。

验收: 至少 2 个 background task；provider/model concurrency 生效；parent wake 无重复。

Lab B - 受控重构

Prometheus 先写计划，Atlas 按计划委派 Junior 修改，Hashline 防止 stale edit，独立测试验证。

验收: Planner 不能改源码；旧 anchor 被拒；goal 要求 tests exit 0。

Lab C - Provider 故障演练

让 primary model 第一次调用返回 simulated 529/rate-limit，Runtime fallback 到第二模型。

验收: proactive resolver 与 reactive fallback 日志可区分；attempt cap 生效。

Lab D - Compaction 恢复

长任务中途强制 compact，再继续执行剩余 Todo。

验收: goal/todo/plan/rules 全部 rehydrate；完整 transcript 可追溯。

Lab E - Team claim race

Lead 创建 3 member，同时竞争两个 shared tasks。

验收: 同一 task 不能双 claim；mailbox ack；worktree cleanup。

Lab F - 第二 Harness

把 Mini-OmO domain core 连接到一个 MockCodexAdapter，而不修改 model/task/team core。

验收: core 不 import host SDK；两 adapter 通过同一 contract tests。

Appendix A. Mini-OmO 核心接口建议

```
class RuntimeContext:
    workdir: Path
    session_id: str
    agent_name: str
    mode: str

class ToolRegistry:
    register(definition, handler, policy=None)
    visible_tools(ctx) -> List[dict]
    execute(call, ctx) -> ToolResult

class HookBus:
    emit(event, payload, ctx) -> HookDecision | None

class ModelResolver:
    resolve(request, availability) -> ResolvedModel

class AgentRegistry:
    get(name) -> AgentConfig
    available(ctx) -> List[AgentConfig]

class TaskDelegator:
    resolve(category=None, subagent_type=None, skills=[]) -> TaskPlan
    run(plan, background=False) -> TaskHandle | Result

class BackgroundManager:
    launch(plan) -> TaskHandle
    cancel(task_id)
    get(task_id)
    shutdown()

class CompletionController:
    on_stop(session) -> StopDecision
```

Appendix B. OmO 真实架构速查

抽象	真实 OmO 对应	一句话职责
Composition Root	createPluginModule()	启动顺序、依赖注入、资源 lifecycle。
Managers	Background/Tmux/SkillMcp/Monitor 等	持有长生命周期 state/resources。
Tools	createTools + Tool Registry	模型可调用能力表面。
Hooks	5-tier composers + direct events	横切 policy/control plane。
Plugin Interface	chat/tool/event/config hooks	OpenCode host adapter boundary。
Agents	builtin-agents + prompt builders	角色、模式、权限、动态 Prompt。
Delegation	delegate-task + delegate-core	category/subagent/model/skill/executor resolution。
Background	background-agent feature	并发、poll、completion、retry、parent wake。
Model Core	model-core	model chain / availability / reasoning policy。
Skills	skills-loader-core + adapters	多 scope discovery / progressive loading。
MCP	built-in + .mcp + skill MCP	外部 capability discovery 与生命周期。
Team	team-core + OpenCode adapter	mailbox/tasklist/state/worktree/tmux。
Hashline	hashline-edit	stale-reference-safe editing。
Config	omo.jsonc + migration	跨 harness/layer/profile 配置。

Appendix C. Production Hardening Checklist

安全

- 所有文件路径做 canonicalization + workspace boundary。
- Tool/MCP 权限在执行层 enforce。
- background/non-interactive ask -> fail closed。
- secrets 不进入 memory/transcript/telemetry。

可靠性

- 每个 background terminal path release slot。
- 子进程/process group 在退出时清理。
- state mutation atomic write；共享 state 加锁。
- fallback/retry/continuation 均有 cap。

Context

- 大输出落盘而不是无限塞 prompt。
- compaction 可追溯，并 rehydrate durable state。
- Dynamic Prompt 只描述当前真实 capability。

可测试性

- LLM/clock/process/filesystem/host API 有 test seam。
- model resolver / permission matcher / prompt sections 尽量 pure。
- 并发与 crash recovery 有 deterministic tests。

可观测性

- 记录 task id/session id/model provenance/attempt。
- doctor 能检查 config/models/tools/dependencies。
- 错误分类而不是只输出 stack trace。

架构

- Manager/Tool/Hook 边界清楚。
- 共享 core 不依赖具体 harness SDK。
- adapter workaround 有 narrow guard、可移除、带 issue reference。

Appendix D. Source Snapshot and References

本教程的 current-behavior 判断以以下源码快照为基准。项目迭代极快，未来默认模型、Hook 数量、配置字段和 package extraction 可能变化；学习架构时优先关注不变量而不是默认值。

项目	Snapshot	重点来源
shareAI-lab/learn-claude-code	eb4307f4e495d2ed22699e1e5682eb55f8076ade (2026-08-12)	s01_agent_loop ... s17_goal_loop; .env.example; requirements.txt
code-yeongyu/oh-my-openagent	530c2c34d9a52ae81f032b2885e059dbbd8a53d1 (2026-08-13)	packages/omo-opencode/src; packages/*-core; docs/reference/configuration.md; docs/guide/orchestration.md
oh-my-openagent latest release near snapshot	v5.0.0-beta.7 (published 2026-08-12)	multi-harness / native / memory runtime evolution

建议源码验证命令:

```
git -C learn-claude-code rev-parse HEAD
git -C oh-my-openagent rev-parse HEAD

git -C oh-my-openagent log -1 --oneline
# 再打开每课列出的 path, 对照当前 dev 是否已迁移到新的 *-core package。
```

结语：你真正应该沉淀的 8 个 Know-How

原则	含义
1. Loop invariant	把 Tool result 喂回模型，直到 Runtime 决定真正完成。
2. Capability surface	Tool/Agent/Skill/MCP 的可用性必须来自 Runtime。
3. Policy outside Prompt	安全与权限写进执行路径和 hooks。
4. Durable state	Todo、Task、Goal、Memory、Team state 分层，不依赖聊天记录。
5. Context routing	按需注入、预算、压缩、rehydrate，而不是“塞更多”。
6. Orchestration depth	Parent/worker 边界清楚；默认有限深度，必要时才 Team。
7. Failure is normal	provider、tool、context、process、worker 都会失败，fallback/retry/cleanup 是一等公民。
8. Adapter boundary	Host 会变化；把可复用逻辑留在 pure/core，把宿主怪癖吸收到 adapter。

END GOAL: 当你完成这 24 课后，你不需要“背会 oh-my-openagent”。你应该能够看到任何 Coding Agent 产品，并从它的 Tool surface、state、hook points、delegation、context、completion policy 和 host adapter 反推出其 Runtime 架构。

